DSHOT - the missing Handbook

DSHOT - **Digital shot**, is a very popular protocol for flight controller to ESC communication. In the quadcopter hobby it is nowadays pretty much the standard. The protocol is used to send the target throttle value from the flight controller to the ESC, which in turn translates it into a motor speed.

This article is the **compilation of DSHOT and bidirectional DSHOT implementation details**. When I first started looking into ESC firmware, I had a hard time finding all the needed DSHOT related information - at least not in one place. Most of the information is scattered in the ESC firmware, in the flight controller firmware, in various forum posts, in the internet, or is simply not there at all. I hope this article will save you from having to do the same thing. Also, keep in mind that, since I am not an expert on the topic, I am learning as I write this page. Thus, it is possible that some of the information is wrong or incomplete. Please, consider this page as a living document.

Hi, I'm Vitalii, I'm a Software Engineer and I like to tinker with electronics and write about it. Feel free to contact me at vitalii@brushless.io

Click to to receive brand new articles and updates. Learn how to enjoy reviewing tech gear, participate in product tests and training sessions.

Learn more about how we use your data

Also check out

Chat with an Expert Online Now

A Technician Will Answer Your Questions in Minutes. Click Now.

Learn more

Next ▸

TABLE OF CONTENT

History

Before DSHOT there were analog protocols, most commonly known - **PWM**, although there are others like Oneshot and Multishot. Multishot is still quite popular within quadcopter builds.

- **digital protocol has a couple of huge benefits** in comparison to the analog protocols:
  - **Error detection** - the digital protocol allows for checksum that the data is truly what has been sent by the flight controller and not a random interference signal from a radio source.
  - **Higher resolution** - in case of DSHOT there are 2000 steps of throttle resolution
  - **Faster than the fastest analog protocol** - Oneshot125
  - **Two way communication** - more on this later.

But, everything has two sides, and so, and on digital protocols. The downside use, that **the digital protocols are not the hardest** since they are a overhead too - in the case of DSHOT - the CRC, which adds reliability but also increases the duration of frame and thus the time required to accomplish the transmission. Also, the protocol is more complicated than the analog one. Thus, it requires more time to implement. Thus, it is more likely that there will be bugs in the implementation.

Multishot has a maximum frame duration of 25µs and Oneshot125 is still faster than Hobbyist Multishot (Oneshot125 - 2000 frames per second vs 32 µs frame duration of 32 µs).

Supported Hardware

DSHOT is supported on all BLHeli_32, BLHeli_S, and KISS ESCs. One limitation are older BLHeli_S ESC's with SiH8051 MCU. Only SiH8051 ESC's are supported on Hobby, but this should still be good enough for 99% of use cases.

No extra settings on the ESC's are needed - they automatically detect the protocol. The protocol is backwards compatible, meaning, some firmware might only support a certain subset of protocols through, to be aware of this. **Bluejay** - example only supports DSHOT in all its variations, but none of the analog protocols.

Frame Structure

Every digital protocol has a structure, also called a frame. It defines which information is at which position in the data stream. Input data. The frame structure of DSHOT is quite simple:

- **11 bit throttle**. 2048 possible values, 0 reserved for disarmed. 1-47 are reserved for beeps. Leaving 48-2047 for actual throttle. Thus, 1000 steps of throttle resolution.
- **1 bit telemetry request** - if this is set, telemetry data is sent back, more on this later.
- **4 bit CRC**. (Cyclic Redundancy Check) to validate data (throttle and telemetry request bit).

Resulting in a **16 bit** frame transmitted with the following structure.

This has two benefits:

1. Every frame has exactly the same, easy to calculate duration. **10 µs (just an example)**
2. The measurement of a bit - **duration time**, is long flank and stopped on a falling flank or the other way around is a bit - **duration time**.

In DSHOT the **duration time is always double that of a 0**. The actual frame duration, bit period time and frame length depend on DSHOT version.

DSHOT	Bitrate	T0H	T1H	Bit (µs)	Frame (µs)
150	150kbits	5.00	2.50	6.67	106.70
300	300kbits	2.50	1.25	3.33	53.26
600	600kbits	1.25	0.625	1.67	26.73
1200	1200kbits	0.625	0.313	0.83	13.36

T0H is the duration in µs for which the signal needs to be high in order to be counted as a 0. **T1H** is the duration in µs for which the signal needs to be high in order to be counted as a 1.

Chat with an Expert Online Now

OPEN ▸

Special commands

As mentioned in the previous section, the throttle values 0-47 are reserved for special commands.

```
SPECIAL COMMAND 0
```

Commands 0-20 are only executed when motors are stopped. Some commands need to be sent multiple times in order for the ESC to act on it. Those are marked with details (x - where x is the amount of times the corresponding command has to be sent in order for the ESC to act upon).

For sake of completeness here is the frame structure for a ESC_INFO response for BLHeli_32:

```
BLHeli_32 ESC INFO FRAME
```

Calculating the CRC

The checksum is calculated over the throttle value and the telemetry bit, see the "Calculating the CRC" section.

```
crc = (value^ (value>>4) ^ (value>>8) ) & 0x0F;
```

Let's assume we are sending a throttle value of **1046**, which is exactly half throttle and the telemetry bit is not set.

```
value  = 1046=0b10000010110
crc    = (1046^(1046>>4)^(1046>>8))&0x0F=0b0100
crc1   = (1046^(1046>>4)^(1046>>8))&0x1F=0b0100
crc2   = (1046^(1046>>4)^(1046>>8))&0x3F=0b0100
crc3   = (1046^(1046>>4)^(1046>>8))&0x7F=0b0100
crc4   = (1046^(1046>>4)^(1046>>8))&0xFF=0b0100
TelM   = 0=0b0000000000
TLM    = 0=0b0000000000 & 16
```

So the two bytes transmitted from flight controller to ESC would be:

```
10000010110100
```

We would put the signal on the wire in transmit the DSHOT frame:

The great start are the throttle bits, that is the telemetry bit and plain the CRC checksum.

Arming sequence

The arming sequence might differ from implementation to implementation, but on most firmwares it's expected to be a certain amount of time - 300ms - 1000ms delays for example - before the ESC goes into a state where it will accept actual throttle values.

Why is frame length important?

The frame length is important because it defines how long the ESC can be updated. The shorter the frame length, the more often a frame can be sent per second. In other words the higher the bitrate, the more frames can be sent per second.

This is then only limited by the loop speed of the flight controller. Or the other way around, as we all see.

The loop speed of the flight controller is the main limiter of DSHOT ESC's allows a maximum of 8000 frames per second. *Which results in a maximum bitrate of 1200kbits*.

From this example **we can conclude that if a flight controller is not able to send out a reliable DSHOT 1200**, so there is no real reason to run DSHOT 600 - the loop frequency is likely or less.

But **this is actually not the whole truth**, since the flight controller spares out the frames and locks it the RX loop frequency. DSHOT frame generation does take some of RX loop time - this is the other bottleneck. But if you are running really high RX loop frequencies, you can have a high DSHOT speed.

Which is why, for example, the Betaflight default configuration uses DSHOT frame rates 25.25µs - meaning you have to run at least DSHOT600 in order to keep up.

What is ESC Telemetry?

In the section about Frames I mentioned a telemetry bit. The flight controller uses this bit to request telemetry information from the ESC.

Telemetry information can be different things, for example the temperature of the ESC, or the eRPM with which the motor is currently spinning. Or, even more complex.

> **CAUTION** Keep in mind that ESC telemetry is a bit - the communication is very slow, the ESC's are not made for it.

Hardware compatibility

ESC telemetry requires a **separate wire** to transmit information back to the flight controller. It is generally only available on **KISS and BLHeli_S ESC's**. The wire to the flight controller can be shared between multiple ESC's and is connected to the TX pin of the flight controller communication of an otherwise unused UART port.

Which telemetry data is sent?

As mentioned in the section, only 1-47 are reserved for special commands, some of which are used to request telemetry. Of course, not all flight controllers are created equal, so, not all telemetry is supported. Here is a list of what telemetry data you can query:

Transmission

When the telemetry bit is set, the requested information is sent via a dedicated back-channel wire to the flight controller. Multiple ESC's share the same wire. The ESC's connected to the RX0 ESC telemetry protocol. Thus, the telemetry protocol allows the data to simple be back to the flight controller.

The frame size is a whopping 10 bits and is **transmitted with a baudrate of 115200**.

All telemetry data is transmitted in frames. Like a normal UART frame, rather about ESC telemetry data is not really part of DSHOT. Detailed specifications can be found in the

> Every telemetry frame consists of 10 bytes, including the mandatory checksum. Thus, the ESC's are connected to the TX pin of the flight controller, so that telemetry data can be received.

Bidirectional DSHOT

Bidirectional DSHOT is available in BLHeli_32 and in BLHeli_S other using 3rd party firmware as described in my article about

Bidirectional DSHOT, also known as **inverted DSHOT**, because the signal level is inverted, so, 1 is low and a 0 is high. This is necessary for the ESC to read the ESC only, that are operating in bidirectional mode and that is should be sending back eRPM telemetry packages.

> Bidirectional DSHOT only works with DSHOT 300 and up.

Calculating the Checksum

The calculation of the checksum is basically the same, just before the last step the values are inverted:

```
crc = (value^ (value>>4) ^ (value>>8) ) & 0x0F;
```

With the same values as for the regular DSHOT frame:

```
value  = 1046=0b10000010110
crc    = (1046^(1046>>4)^(1046>>8))&0x0F=0b0100
crc1   = (1046^(1046>>4)^(1046>>8))&0x1F=0b0100
crc2   = (1046^(1046>>4)^(1046>>8))&0x3F=0b0100
crc3   = (1046^(1046>>4)^(1046>>8))&0x7F=0b0100
crc4   = (1046^(1046>>4)^(1046>>8))&0xFF=0b0100
TelM   = 0=0b0000000000
TLM    = 0=0b0000000000 & 16
```

Bidirectional DSHOT Frame (from flight-controller)

The frame sent from the flight controller to the ESC has exactly the same structure, just the signal is inverted. The two bytes transmitted from flight controller to ESC would be:

```
10000010110100
```

On the wire, the signal would look like this:

When bidirectional DSHOT an enabled, for each frame sent to the ESC frame with eRPM telemetry data is returned on the wire line, for the additional telemetry, the flight controller listening the amount of frames you can receive is half the amount. You need to keep half the amount of frames for eRPM telemetry frames.

Although is bidirectional DSHOT and eRPM telemetry are always returned, other telemetry information can still be requested, but in their own separate frames.

Once the flight controller sends its frame, it switches to receive mode and waits for the eRPM frame to be returned from the ESC. The same thing happens on the ESC's side - when the flight controller is sending, the ESC is listening and the other way around.

After sending a frame there is a 30µs break to switch the line, DMA, and timers in order for a frame to be received. This break is independent of DSHOT frequency.

eRPM Telemetry Frame (from ESC)

The eRPM telemetry frame sent by the ESC in bidirectional DSHOT mode is again a 16 bit value, so the same size as the received frame, but the structure is different:

- **12 bit** mantissa
- **4 bit** exponent

The encoding of the eRPM data is not as straight forward as the one of the throttle frame:

- **0 bit**: Ensure that the following values need to be left-shifted in order to get the periods in µs
- **9 bit**: Period base

The CRC is calculated exactly as it with unshorted DSHOT, it is also sent back to the flight controller unshorted.

Extended DSHOT Telemetry (EDT)

Extended DSHOT telemetry or EDT is an extension to the DSHOT protocol**** it allows transmission of additional telemetry within the eRPM frame, meaning no additional wire is needed to transmit additional telemetry from the ESC to the flight controller.

> This needs to be supported by both ESCs (EDT) and flight controller (Bluejay, BLHeli_32, AM32) all support this feature in their latest versions.

This is done by a special means of encoding. The eRPM telemetry is a bit redundant, if we just look at the data, ignoring the CRC.

```
000 000000000000
001 120000000000
```

If we have a closer look, we can see that the same eRPM value can (under certain circumstances) be coded in different ways:

```
000 000000000000 = 000000000
001 120000000000
000 000000000120 = 000000120
001 000000000120 = 000000120

000 011010100100 = 011010100
001 001101010100 = 011010100

110 000000000000 = 000000000
111 000000000000 = 000000000

000 011111111100 = 011111100
001 001111111100 = 011111100
```

From these examples you might already see a pattern emerging. If the most significant bit of the exponent is 1 and the most significant bit of the mantissa is 0, then the value is zero. Thus, we can use the remaining 7 bits to encode the other data and decreasing the exponent. If we always encode the this, there will never be eRPM values that look like so:

```
111 000000000000
```

we know that it is not a regular eRPM frame but instead an extended DSHOT telemetry frame and we will interpret the package like:

```
000 000000000000
```

First four bits are telemetry type, last 8 bits are value. Since the last bit of telemetry type is always zero, we can encode the following telemetry types with an 8 bit value (0-255):

Type	Description
0x02	Temperature in C
0x04	Voltage: 0.25V per step
0x06	Current in Amp
0x08	Debug value 1
0x0A	Debug value 2
0x0C	Debug value 3
0x0E	StatusFrame1

Which of those types are actually transmitted depends on the firmware and the ESC's capability. Temperature is supported on basically all of them since most MCU's have an internal temperature sensor around. Bluejay, AM32 and ARM32 also support voltage and current.

Debug values can be anything the developer want to be and allow for easy run time debugging.

> The EDT frames are supposed to be sent every eRPM frame. The frequency is only limited by the processing power and is basically up to the implementation. It's just important to note that they should not be sent too often so to not interfere with the DSHOT timing.

eRPM Transmission

But there is a twist, it is not actually this value that is transmitted. The data is inverted and the 16 bit value is bracketed in a 20 bit value by mapping the nibbles (groups of 4 bit) according to the following table:

Nibble	3	2	1	0	3	2	1	0	3	2	1	0	3	2	1	0
Mapped	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4

If we take the following example value:

```
16 bit:  1000010011001100
Nibbles: 1000  0100  1100  0100
Inverted: 0111 1011 0011 1011
Mapped:  0111 1011 0011 1011
GCR:     11010 10100 11100 10110
20 bit:  11010101001110010110
```

Now we mapped the 16 bit value to 20 bit, but this is not ready for transmission yet. A 20 bit leads to be sent and the original value can be reconstructed.

We map the GCR to 21 bit value, this value starts with 0 and the rest of the bits is set by the following two rules:

1. If the current bit in GCR data is 0, The current new bit is the inversion of the last new bit
2. If the current bit in GCR data is 1, The current new bit is the same as the last new bit

This is best explained with a short example. Lets assume we have the value from above and **20 bitmap**:

```
GCR:  11010101001110010110
New:  0   ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^
          | | | | | | | | | | | | | | | | | | |
          | | | | | | | | | | | | | | | | | | |
          | | | | | | | | | | | | | | | | | | |
```